作业1：Hello world 的两种运行方式

实验日期：2026-10-16。实验对象：8086 指令集、16 位 DOS 程序。

1. 实验要求与完成情况

按照作业截图与《hello剖析.pdf》完成以下任务：

1. 使用“源码 → MASM 汇编 → LINK 链接 → EXE 运行”的传统流程，提交源码和 EXE。
2. 用 DEBUG 的 `u` 命令观察最终 EXE 的机器码与源码的对应关系。
3. 使用 DEBUG 的 `e` 命令直接写入字符数据和机器码，设置入口并执行，提交操作文档。
4. 理解数据段、代码段、栈、程序入口和常用寄存器。

本次两份 EXE 均为 0 警告、0 汇编错误，链接成功；传统方式与直接写内存方式均输出 `Hello world!` 并正常退出。自动检查结果为 23/23 项通过，详见 验证结果.json。

2. 环境搭建与使用

组件	本机配置	用途
主机	Windows 11，64 位	编辑源码、保存实验文件
DOSBox-X	2026.10.01，便携版	运行 16 位 DOS 工具和程序
MASM	Microsoft Macro Assembler 5.00	将 ASM 汇编成 OBJ，生成 LST 列表
LINK	Microsoft Overlay Linker 3.60	生成可执行 EXE 和 MAP 映射
DEBUG	FreeDOS DEBUG 2.50，命令名为 debug	写内存、反汇编、查看寄存器、单步执行

课程根目录为 `D:\Homework\6\HUI`，课程入口为根目录中的 `汇编课程环境.lnk`。配置文件 `dosbox-course.conf` 将根目录下的 `dos` 映射为 DOSBox 的 D 盘，并进入 `D:\WORK>`。

源码的实际编辑目录是 `D:\Homework\6\HUI\dos\work`。MASM、LINK、DEBUG 在 DOSBox 的 `D:\BIN` 中，已加入 DOS 的 PATH。DEBUG 的 Windows 实际位置为 `D:\Homework\6\HUI\dos\bin\DEBUG.COM`；`.COM` 是调试器自己的可执行文件格式，它可以调试本次的 `.EXE` 文件。

双击根目录的 `运行作业1.cmd` 可依次演示三次输出；手工操作时，每次只输入一条命令并等待提示符。代码使用 ASCII、无 BOM，DOS 文件名不超过 8 个字符。

3. 传统方式：源码、汇编、链接和运行

3.1 完整源码

A1LEA.ASM 与讲义中的 LEA 版本一致，并明确增加 256 字节栈空间：

```
; Assignment 1: Hello world, using LEA (MASM 5.0 / 8086 DOS).
.MODEL SMALL
.STACK 100H
.DATA
Hello DB 'Hello world!',0DH,0AH,'$'
.CODE
START:
    MOV AX,@DATA
    MOV DS,AX
    LEA DX,Hello
    MOV AH,09H
    INT 21H
    MOV AX,4C00H
    INT 21H
END START
```

第二份 A1OFF.ASM 把 `LEA DX,Hello` 换成 `MOV DX,OFFSET Hello`，其他逻辑相同。两种写法都将字符串偏移放入 DX。

3.2 手工执行命令

在 `D:\WORK>` 下逐行输入：

```
masm A1LEA.ASM,A1LEA.OBJ,A1LEA.LST;
link A1LEA.OBJ,A1LEA.EXE,A1LEA.MAP;
A1LEA
masm A1OFF.ASM,A1OFF.OBJ,A1OFF.LST;
link A1OFF.OBJ,A1OFF.EXE,A1OFF.MAP;
A1OFF
```

每个 EXE 的运行结果均为一行 `Hello world!`，随后回车换行。原始输出是 14 字节：

```
48 65 6C 6C 6F 20 77 6F 72 6C 64 21 0D 0A
```

字符串末尾的 `$` 是 AH=09H 服务的结束标记，不会被输出。`0DH` 是回车，`0AH` 是换行。

3.3 DEBUG 的 u 命令与真实反汇编

```
debug A1LEA.EXE
u cs:0 l12
q
debug A1OFF.EXE
u cs:0 l11
q
```

`u` 从给定地址反汇编；`l12` 和 `l11` 分别限定 12H、11H 字节。DEBUG 数字默认按十六进制解释。以下取自本次真正加载 EXE 后的 DEBUG 输出，左起依次为“段:偏移、机器码、汇编指令”。段地址会随环境变化。

LEA 版本：

```
0E00:0000 B8010E            MOV     AX,0E01
0E00:0003 8ED8              MOV     DS,AX
0E00:0005 8D160200          LEA     DX,[0002]
0E00:0009 B409              MOV     AH,09
0E00:000B CD21              INT     21
0E00:000D B8004C            MOV     AX,4C00
0E00:0010 CD21              INT     21
```

MOV OFFSET 版本：

```
0E00:0000 B8010E            MOV     AX,0E01
0E00:0003 8ED8              MOV     DS,AX
0E00:0005 BA0200            MOV     DX,0002
0E00:0008 B409              MOV     AH,09
0E00:000A CD21              INT     21
0E00:000C B8004C            MOV     AX,4C00
0E00:000F CD21              INT     21
```

两种取地址写法的区别：`8D 16 02 00` 是 `LEA DX,[0002]`，共 4 字节；`BA 02 00` 是 `MOV DX,0002`，共 3 字节。本例最终均得到 `DX=0002H`。LEA 计算有效地址，不读取字符串内容；OFFSET 是汇编期地址表达式，链接后成为 MOV 的立即数。两者不是把字符串装入 DX。

本次 MAP 显示 LEA 版代码长 12H 字节，数据起始线性相对位置为 00012H，而 DGROUP 段基址为 00010H，因此 Hello 在 DS 内的偏移为 `00012H-00010H=0002H`。不能根据源码中“第一个变量”就认定最终 EXE 中偏移一定为 0000H，应结合最终链接和反汇编结果判断。

3.4 单步观察寄存器

重新加载 `debug A1LEA.EXE` 后，输入 `tm 1`，再连续输入四次 `t`，最后输入 `r` 和 `d ds:dx l0f`。四次单步依次执行 MOV AX、MOV DS、LEA、MOV AH。本次得到：

```
AX=0901  DX=0002  DS=0E01  CS=0E00  IP=000B
DS:DX = 0E01:0002
```

这里 `AH=09H`，AL 仍保留原先 AX 的低字节 01H；只改 AH 不等于把整个 AX 赋成 0009H。`DS:DX` 处是完整的 `Hello world!`、回车换行及 `$`。再输入 `g` 后输出文本，并显示 `Program terminated normally (0000)`；`q` 退出 DEBUG。

4. 直接写内存：不经过 MASM、LINK 执行机器码

4.1 地址安排

在 DOS 提示符下单独输入 `debug`，不加载 EXE。DEBUG 为被调试程序准备内存和初始寄存器。将 16 字节代码放在当前 `CS:0100H`，将 15 字节字符数据放在 `CS:0200H`；两者不重叠。代码前两条将 DS 初始化为 CS，使 `DS:0200H` 指向同一段中的字符数据。

本实验使用当前 CS，不照抄讲义截图中的 `076A`、`076B` 段地址。讲义第 2 页的 `BA 02 00` 表示 `DX=0002H`，如果把字符串从偏移 0000H 写入，这两个地址便不一致。本次字符串写入 0200H、机器码中的 DX 也为 0200H，保证地址匹配。

4.2 可照做的完整操作

先在 DOS 提示符输入：

```
debug
```

看到 DEBUG 的 `-` 提示符后，逐行输入下面的内容；不要把提示符 `-` 本身输入进去。在 `r ip` 之后，下一行 `100` 是对新 IP 值的回答。

```
e cs:100 8c c8 8e d8 ba 00 02 b4 09 cd 21 b8 00 4c cd 21
e cs:200 48 65 6c 6c 6f 20 77 6f 72 6c 64 21 0d 0a 24
d cs:200 l0f
u cs:100 l10
r ip
100
r
g=100
q
```

前两条 `e` 命令分别把机器码和 ASCII 数据直接写入内存。`d` 检查数据，`u` 检查代码，`r ip` 设置入口偏移，`g=100` 明确从当前 CS 的 0100H 执行，`q` 返回 DOS。第一条写入后不要先运行，必须先完成第二条数据写入。

也可以在 DOS 提示符下输入 `debug < A1MEM.DBG`，按相同顺序自动执行这份 DEBUG 命令文件。此路径仍由 DEBUG 的 `e` 写内存执行，不调用 MASM 或 LINK。

4.3 机器码逐条解释

```
0DA6:0100 8CC8              MOV     AX,CS   ; 复制当前代码段到 AX，避免硬编码运行时段地址
0DA6:0102 8ED8              MOV     DS,AX   ; DS=CS，让字符串与代码使用同一个段
0DA6:0104 BA0002            MOV     DX,0200  ; DX=0200H，必须与字符串写入偏移 0200H 一致
0DA6:0107 B409              MOV     AH,09  ; AH=09H，选择显示字符串服务
0DA6:0109 CD21              INT     21  ; 输出 DS:0200H 处以 $ 结束的字符串
0DA6:010B B8004C            MOV     AX,4C00  ; AX=4C00H，正常退出并返回 0
0DA6:010E CD21              INT     21  ; 结束程序并返回 DOS
```

`BA 00 02` 中的 16 位立即数按低字节在前存储，所以表示 0200H；`B8 00 4C` 表示 AX=4C00H。字符串写入数据为：

```
48 65 6C 6C 6F 20 77 6F 72 6C 64 21 0D 0A 24
 H  e  l  l  o     w  o  r  l  d  ! CR LF  $
```

4.4 本次执行记录

DEBUG 实际分配的段为 0DA6H。依次执行四次 `t` 后，刚好停在输出中断前，本次寄存器为：

```
AX=09A6  DX=0200  DS=0DA6  CS=0DA6  IP=0109
```

因此字符地址为 `DS:DX=0DA6:0200`，实模式物理地址按 段值×10H+偏移 计算，即 `0DA6H×10H+0200H=0DC60H`。这些数值是本次记录，重新启动时段地址可能不同。

执行 `g` 后的实际结果：

```
Hello world!

Program terminated normally (0000)
```

完整记录见 直接写内存调试记录。这次代码由手工写入内存执行，没有经过汇编和链接，也没有另生成 EXE。

5. 程序基本结构

源码结构	含义及运行时关系
.MODEL SMALL	选择 small 模型，通常使用一个代码段和一个数据组，各自的近地址空间最多 64 KiB
.STACK 100H	预留 256 字节栈，链接后由 EXE 头给出初始 SS:SP
.DATA / Hello DB ...	定义数据；DB 按字节存储字符串、控制字符和结束标志
.CODE / START:	定义代码段和入口标签，CPU 从入口开始执行指令
MOV AX,@DATA / MOV DS,AX	在运行时初始化 DS；EXE 启动时不能假定 DS 已指向自己的数据组
LEA DX,Hello	将字符串的有效偏移装入 DX，与 DS 共同组成地址
MOV AH,09H / INT 21H	按 DOS 接口约定，以 DS:DX 为参数显示 $ 结束的字符串
MOV AX,4C00H / INT 21H	退出程序，返回码为 0
END START	汇编指示，指定源文件结束及入口；不是 CPU 指令，也不会代替 DOS 退出调用

`.MODEL`、`.STACK`、`.DATA`、`.CODE`、`DB`、`END` 等属于汇编指示或数据定义；`MOV`、`LEA`、`INT` 等才是实际执行的指令。代码、数据、栈按用途区分；直接写入实验为简短示例将代码和数据放在同一段的不同偏移，仍必须保证数据地址与取指入口正确。

6. 常用寄存器作用

寄存器	主要作用	本实验中的对应关系
AX / AH / AL	AX 是 16 位通用寄存器；AH、AL 分别是高、低 8 位	用 AX 中转段地址；AH=09H 显示字符串；AX=4C00H 正常退出
BX	通用寄存器，也可作基址参与内存寻址	本程序未显式使用，不能把它理解为固定保存某一种数据
CX	通用寄存器；LOOP 和部分重复指令使用的计数寄存器	本程序未用循环，调试器显示的初值不是程序指定的计数
DX	通用寄存器，也作为部分 DOS 服务的参数寄存器	AH=09H 时，DX 保存字符串在 DS 段中的偏移；并不保存整个字符串
SI / DI	源、目的变址寄存器，可参与寻址和串操作	本程序未显式使用
BP	基址寄存器，常用于访问栈内参数；使用 BP 的常见寻址默认段为 SS	本程序未显式使用
SP	栈顶偏移，与 SS 一起定位栈顶	EXE 的栈空间由 .STACK 100H 定义，本次初始 SP=0100H
CS	代码段寄存器	与 IP 一起指向将执行的代码；DEBUG 的 u cs:0 从代码入口反汇编
IP	指令指针，保存下一条指令的偏移	传统 EXE 入口偏移为 0000H；直接写入的代码入口选为 0100H
DS	默认数据段寄存器	传统方式由 MOV AX,@DATA / MOV DS,AX 初始化；直接方式令 DS=CS
ES	附加段寄存器，常见于串指令的目的地址 ES:DI	本程序未显式使用
SS	栈段寄存器	与 SP 一起管理栈；中断执行也需要有效的栈
FLAGS	标志寄存器，记录运算状态及控制状态	常用 ZF 表示结果为零，CF 表示无符号进位/借位，OF 表示有符号溢出；本程序未按标志分支

16 位寄存器保存数值、偏移或段值，具体含义取决于指令和服务接口。要区分 `CS:IP` 的取指地址、`DS:DX` 的本次字符串参数地址与 `SS:SP` 的栈地址。段地址和偏移地址都需要参与定位，单独看 DX 不能确定字符的物理地址。

7. 实验结论与提交内容

传统方式由汇编器把指令转成机器码、由链接器安排段和重定位，再由 DOS 装入 EXE；直接写内存方式由 DEBUG 的 `e` 命令完成机器码与数据写入，再从指定 CS:IP 执行。两种方式最终都让 CPU 执行机器码，并以同样的 DOS INT 21H 服务输出文本和退出。

提交材料包括：

- A1LEA.ASM 与 A1LEA.EXE：主要传统版本。
- A1OFF.ASM 与 A1OFF.EXE：讲义第二种取地址写法。
- A1MEM.DBG：直接写内存的完整 DEBUG 命令。
- 本文档和同名 PDF：包含直接写入操作、程序结构、寄存器与实验结论。
- 反汇编逐行注释、运行与调试记录和 验证结果。

依据：用户提供的作业截图及《hello剖析.pdf》第 1–2 页；反汇编、寄存器和输出采用本机实际记录。